SOFTABLITZ 2026

TEAM: SPARK

TEAM ID

128

TEAM INFO

Member 1: Chetan Srivastav (20243082, CSE, chetan7971)

DESCRIPTION OF THE PROBLEM

Dimensional incursions bring powerful characters from disconnected fictional worlds into a single arena. Mystic Kombat turns this premise into an offline, local 2D fighting game where players choose a champion, learn distinct combat styles, build Mystic Energy (Ki) by dealing and absorbing damage, and spend a full meter on a dramatic Ultimate Incantation
.The engineering challenge is to deliver responsive, readable combat and a complete match loop while keeping timing,collisions, round transitions and effects stable on a single desktop.

App name: MYSTIK COMBAT

PROCESS FLOW OF THE APP

Game Launch: Initialize game resources, load character configurations, and retrieve saved settings and high scores.

Main Menu: Allow players to access Local Duel, Training Mode, Arcade Mode (planned extension), settings, or quit.

Character Selection: Players select fighters with distinct health, speed, damage, special moves, and ultimate attacks.

Match Initialization: Reset health, Ki meters, timers, round scores, input buffers, and fighter positions.

Combat: Players move, jump, crouch, attack, block or use special abilities. The combat engine detects hits, applies damage, calculates knockback, and updates Ki meters.

Ultimate Incantation: When the Ki meter is full, the player can activate a cinematic ultimate attack with visual effects and enhanced damage.

Round Resolution: A round ends when a fighter is knocked out or the timer expires. The game follows best-of-three knockout rules.

Result Screen: Display the winner, match statistics, and available actions.

Rematch or Exit: Players can restart the match, return to the main menu, or quit.

Local Persistence: Save settings and Arcade Mode high scores locally without requiring an internet connection.

FEATURES ANNOUNCEMENT

Core Features

- Offline Desktop Gameplay: Complete game loop from the main menu to combat and results, requiring no internet connection.
- Playable Fighters: At least two distinct characters with unique health, movement speed, attack damage, special abilities, and ultimate attacks.
- Responsive Movement: Walking, jumping, crouching, arena boundary collision, and automatic opponent-facing.
- Combat System: Light and heavy attacks using hitbox and hurtbox detection, attack cooldowns, hit-stun, and knockback.
- Health and Round Management: Health bars, round timer, knockout detection, and best-of-three matches.
- Local Multiplayer: Two players can compete using shared keyboard controls.
- Mystic Energy (Ki): Energy accumulates through damage dealt and damage received.
- Ultimate Incantation: A high-impact special attack unlocked by a full Ki meter, featuring cinematic flashes, hit-stop, slow motion, particles, and screen shake.
- Character Animation: Core states including idle, walking, attacking, taking damage, and knockout.
- Atmospheric Arena: A visually engaging arena with background music, sound effects, and ambient effects.
- Complete Game Loop: Main menu, character selection, combat, result screen, rematch, and quit functionality.

Advanced Features

- Expanded Roster: Four or more playable characters inspired by different fictional archetypes.
- Signature Special Moves: Two special moves per character, such as projectiles, teleports, shields, or gadgets.
- Combo System: Input buffering, cancel windows, combo counter, and damage scaling to prevent infinite damage loops.
- CPU Opponent: Finite-state machine AI with selectable easy, medium, and hard difficulty levels.
- Arcade Mode: A sequence of at least four opponents, culminating in a boss encounter, with local high-score tracking.
- Training Mode: Display active hitboxes, hurtboxes, and recent input history to help players understand combat mechanics.
- Multiple Arenas: Two or more arenas with parallax scrolling, ambient particles, and distinct visual themes.
- Grimoire-Driven Architecture: Load character statistics and move frame data from external JSON configuration files, with runtime validation and a basic frame-data inspector.

External libraries/APIs

1. Godot Engine 4

Purpose: Primary game engine for developing the offline 2D fighting game.

Why chosen:

- Free and open-source under the MIT license.
- Built-in support for 2D rendering, physics, collision detection, animation, audio, particles, and keyboard/gamepad input.
- Scene-based architecture enables modular development and reusable game components.
- Supports exporting standalone desktop builds.
- Strong official documentation and an established community.

Trade-offs: Godot requires learning its scene and node architecture. However, it eliminates the need to implement low-level rendering, physics, and audio systems independently.

Website: https://godotengine.org/

2. GDScript

Purpose: Implement combat logic, movement, fighter states, game rules, AI, and UI interactions.

Why chosen:

- Native scripting language designed for Godot.
- Simple syntax and rapid development workflow.
- Direct access to engine features without additional runtime dependencies.
- Suitable for implementing state machines, combat rules, and gameplay logic.

Trade-offs: GDScript is primarily associated with Godot, so it is less portable to unrelated engines than general-purpose languages.

Documentation: https://docs.godotengine.org/

3. Godot Resources and JSON

Purpose: Store character attributes, move definitions, attack damage, timing values, and configuration data outside the core combat logic.

Why chosen:

- JSON is human-readable and easy to edit, compare, and validate.

- Godot Resources support typed, editor-friendly configuration.
- External data allows new moves and characters to be added without rewriting the combat engine.
- Validation can detect missing fields, invalid values, and incompatible configuration versions.

Trade-offs: JSON requires explicit validation and error handling. Godot Resources may be more convenient for configurations edited directly in the engine.

4. Built-in Godot Animation, Audio, and Particle Systems

Purpose: Implement character animations, background music, sound effects, visual effects, and ultimate attack cinematics.

Why chosen:

- Built into the engine, reducing third-party dependencies.
- Supports animation timelines, audio playback, particles, and visual effects.
- Simplifies integration with combat events.
- Reduces setup and compatibility risks within the one-week development timeline.

Trade-offs: Advanced effects may require optimization to avoid frame-time spikes on lower-end systems.

5. Kenney, OpenGameArt, and Freesound

Purpose: Obtain free or openly licensed sprites, arena assets, and sound effects.

Why chosen:

- Reduces the time required to create every asset from scratch.
- Provides reusable resources suitable for prototypes and indie games.
- Allows more development time to be spent on combat mechanics and reliability.

Trade-offs: Asset licenses vary. Each selected asset will be checked for attribution requirements and permitted usage.

Core Engineering Architecture

1. Infrastructure Layout and Data Flow

Mystic Kombat follows an offline-first, modular desktop architecture. Since the game does not require online services, the executable contains the gameplay systems, resource management, rendering, and local persistence logic. No remote server, API gateway, or cloud database is required during gameplay.

Architectural Components

1. Input Layer: Captures keyboard and gamepad inputs and converts them into gameplay commands.
2. Game Manager: Controls transitions between the main menu, character selection, combat, round results, and match results.
3. Combat Engine: Processes movement, attack states, hitbox and hurtbox collisions, damage, knockback, hit-stun, and Ki generation.
4. Character System: Maintains each fighter's health, speed, damage attributes, current state, and special abilities.
5. Grimoire Configuration: Loads and validates character attributes and move definitions from external data files.
6. Rendering and Effects: Displays characters, health bars, Ki meters, animations, arena backgrounds, particles, and cinematic effects.
7. Audio Manager: Controls background music and sound effects triggered by combat events.
8. Local Save Service: Stores settings and high scores in versioned local files.

Data Flow

1. At startup, the game loads configuration files, required assets, and saved settings.
2. The input layer captures player commands and passes them to the relevant fighter controller.
3. Fighter state machines determine whether movement, attacks, or special moves are valid.
4. The combat engine checks active hitboxes against opponent hurtboxes and calculates damage and knockback.
5. Health and Ki meters are updated, and combat events trigger animations, audio, and visual effects.
6. The round manager checks for knockouts or timer expiry and determines the winner.
7. The game transitions to the next round or result screen.
8. The save service writes settings and high scores when necessary.

2. Resource Management, Reliability, and Failure Prevention

Although the game is offline, performance and timing problems can still occur due to excessive visual effects, unbounded object creation, faulty state transitions, or poorly managed input.

The following engineering choices address these risks:

- Controlled simulation timing: Use a fixed-step gameplay simulation with bounded catch-up steps to reduce frame-rate-dependent combat behavior.
- Explicit state management: Use defined game and fighter states to prevent invalid transitions, stuck attacks, or incorrect round resets.
- Collision management: Separate attack hitboxes from fighter hurtboxes and track which targets have already been hit during an attack.
- Resource management: Preload essential assets, reuse frequently spawned effects where practical, and limit excessive particle generation.
- Configuration validation: Validate character statistics, move timing, damage values, and resource references before combat begins.
- Safe local persistence: Version save files, validate loaded data, and use temporary-file replacement where supported to reduce corruption risk.

- Defensive input handling: Clear stale input buffers when necessary, particularly during knockouts, round transitions, and menu changes.
- Testing: Test simultaneous attacks, arena boundaries, repeated ultimate activation, rapid rematches, timer expiry, and window focus loss.

3. The Naive Approach vs. the Proposed Engineering Approach

Naive/basic approach	Proposed engineering approach	Why it wins
Place all gameplay logic, UI, input, and round handling in one large script.	Separate responsibilities into GameManager, Fighter, CombatSystem, RoundManager, InputBuffer, and supporting components.	Improves maintainability and makes changes easier to isolate and test.
Apply damage every frame while an attack overlaps an opponent.	Use explicit attack phases and per-attack hit registration.	Prevents a single attack from repeatedly applying damage unintentionally.
Make movement and timers depend directly on rendered frames.	Use a fixed-step simulation for gameplay timing.	Reduces inconsistent combat behavior across different frame rates.
Hard-code all character and move properties.	Load validated move data from external configuration files.	Simplifies balancing and adding new fighters without changing core combat logic.
Overwrite the only save file directly without validation.	Use versioned save data and safe file replacement.	Reduces the risk of corrupted settings and high scores.
Add visual effects without resource limits.	Bound particle counts, manage effect lifetimes, and reuse resources where appropriate.	Helps maintain stable frame rates during cinematic attacks.

Class diagram

